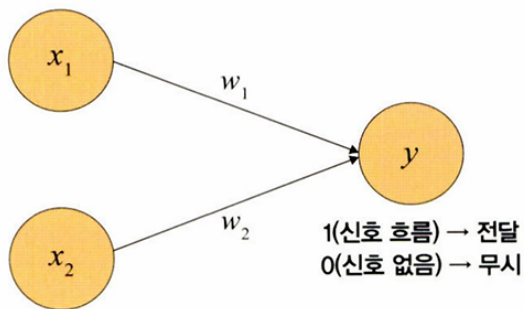


Chapter04. 딥러닝

4.1 인공 신경망의 한계와 딥러닝 출현

퍼셉트론

▼ 그림 4-1 퍼셉트론 원리



- 인공 신경망에서 이용하는 구조(입력층, 출력층, 가중치로 구성된 구조)
- 다수의 신호를 입력 받아, 하나의 신호 출력
 - 흐른다/안 흐른다(1또는0)는 정보를 전달하는 원리로 작동

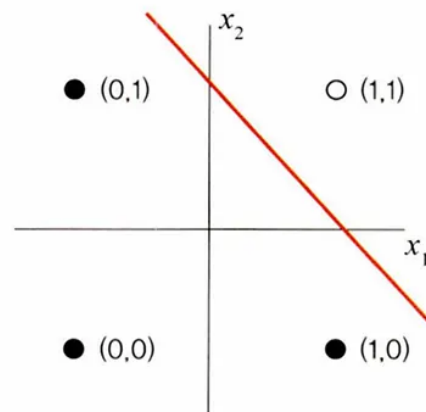
다음은 컴퓨터가 논리적으로 인식하는 방식을 알아보기 위해 논리 게이트로 확인해본다.

AND 게이트

모든 입력이 '1'일 때 작동

▼ 표 4-1 AND 게이트

x_1	x_2	y
0	0	0
1	0	0
0	1	0
1	1	1

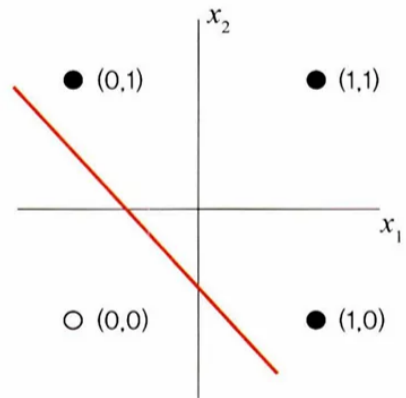


OR 게이트

둘 중 하나만 '1'이거나 둘 다 '1'일 때 작동

▼ 표 4-2 OR 게이트

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	1



XOR 게이트

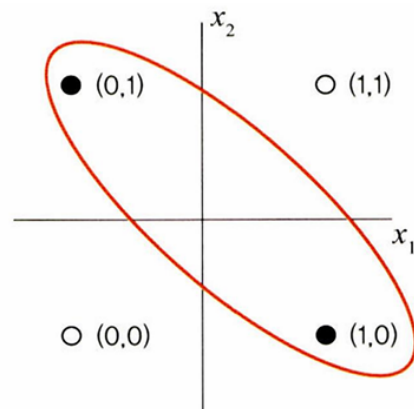
두 개 중 한 개만 '1'일 때 작동하는 논리 연산

- 데이터가 비선형적으로 분리 돼, 분류가 어려움

⇒ 단층 퍼셉트론에서는 AND, OR 연산 학습 가능, XOR 학습 불가능

▼ 표 4-3 XOR 게이트

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	0



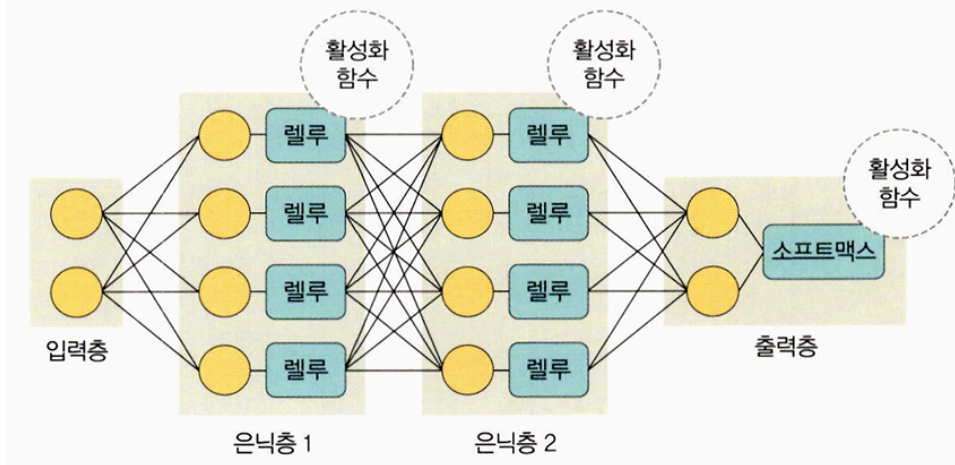
4.2 딥러닝 구조

4.2.1 딥러닝 용어

딥러닝 구조

- 입력층, 출력층, 두 개 이상의 은닉층

▼ 그림 4-5 딥러닝 구조



딥러닝 구성요소

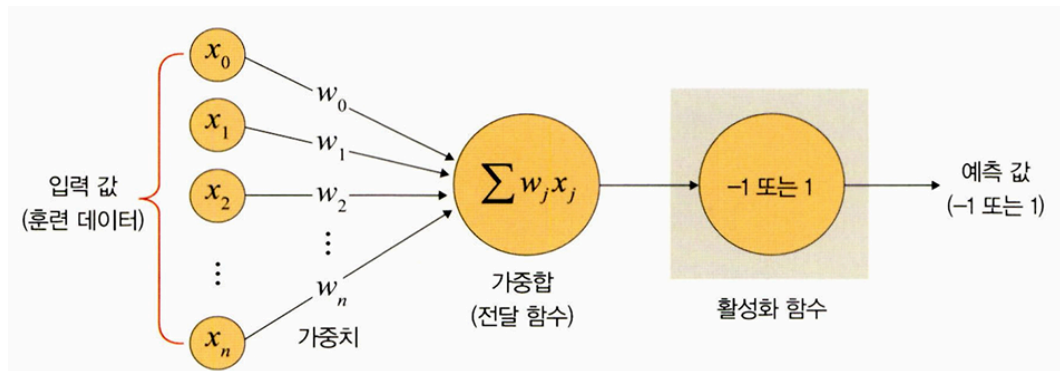
구분	구성 요소	설명
층	입력층(input layer)	데이터를 받아들이는 층
	은닉층(hidden layer)	모든 입력 노드부터 입력 값을 받아 가중합을 계산하고, 이 값을 활성화 함수에 적용하여 출력층에 전달하는 층
	출력층(output layer)	신경망의 최종 결과값이 포함된 층

구분	구성 요소	설명
가중치(weight)		노드와 노드 간 연결 강도
바이어스(bias)		가중합에 더해 주는 상수로, 하나의 뉴런에서 활성화 함수를 거쳐 최종적으로 출력되는 값을 조절하는 역할을 함
가중합(weighted sum), 전달 함수		가중치와 신호의 곱을 합한 것
함수	활성화 함수(activation function)	신호를 입력받아 이를 적절히 처리하여 출력해 주는 함수
	손실 함수(loss function)	가중치 학습을 위해 출력 함수의 결과와 실제 값 간의 오차를 측정하는 함수

가중치

입력 값이 연산 결과에 미치는 영향력을 조절하는 요소

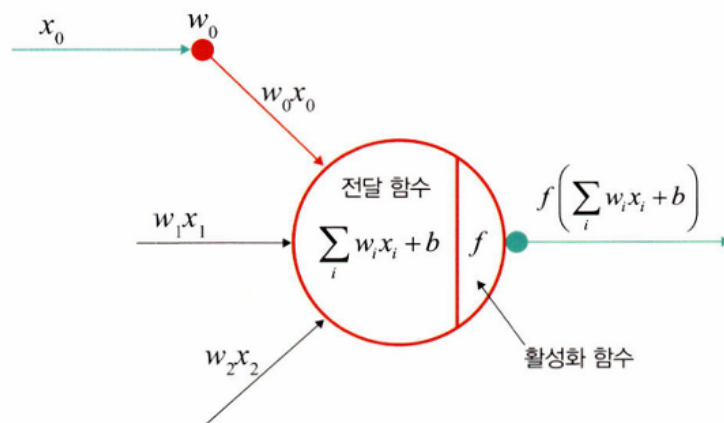
- ex) w_1 이 0.001이면, x_1 이 크더라도 $x_1 * w_1$ 값은 거의 0임



가중합/전달 함수

각 노드에서 들어오는 신호에 가중치를 곱한 값들의 합계

- 노드의 가중합이 계산되면 이 가중합을 활성화 함수로 보내기 때문에 전달함수 라고도 함



- 가중합을 구하는 공식

$$\sum_i w_i x_i + b$$

(w : 가중치, b : 바이어스)

활성화 함수

전달 함수에서 전달 받은 값을 출력할 때 일정 기준에 따라 출력 값을 변화시키는 비선형 함수

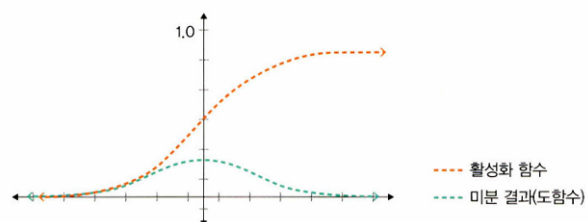
시그모이드

선형 함수의 결과를 0~1 사이에서 비선형 형태로 변형

- 로지스틱 회귀와 같은 분류 문제를 확률적으로 표현하는 데 사용
- 단점: 기울기 소멸 문제

$$f(x) = \frac{1}{1 + e^{-x}}$$

시그모이드 수식

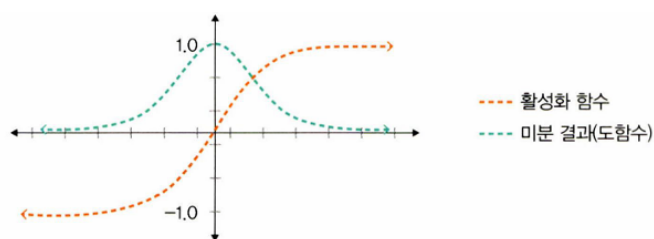


시그모이드 활성화 함수와 미분 결과

하이퍼볼릭 탄젠트 함수

선형 함수의 결과를 -1 ~ 1 사이에서 비선형 형태로 변형

- 장점: 시그모이드의 단점인 결과값 평균이 0이 아닌 양수 편향 문제를 개선
- 단점: 기울기 소멸 문제 여전히 발생



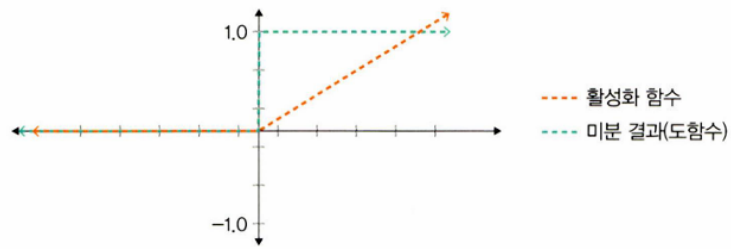
하이퍼볼릭 탄젠트 활성화 함수와 미분 결과

렐루 함수

입력이 음수일 때는 0 출력, 양수일 때는 x 출력

- 일반적으로 은닉층에서 사용
- 장점: 경사하강법 영향X → 학습속도가 빠름, 기울기 소멸 문제X

- 단점: 음수 값을 입력받으면 0을 출력해서 학습능력 ↓ ⇒ 해결: 리키 렐루 함수 사용



렐루 활성화 함수와 미분 결과

소프트맥스 함수

입력 값을 0~1 사이에 출력되도록 정규화하여 출력 값들의 총합이 항상 1이 되도록 함

- 딥러닝에서 출력 노드의 활성화 함수로 많이 사용

$$y_k = \frac{\exp(a_k)}{\sum_{i=1}^n \exp(a_i)}$$

소프트맥스 함수 수식

- $\exp(x)$ 는 지수 함수
- n 은 출력층의 뉴런 개수, y_k 는 그 중 k 번째 출력
⇒ 분자는 입력 신호 a_k 의 지수 함수, 분모는 모든 입력 신호의 지수 함수 합으로 구성

#렐루 함수와 소프트맥스 함수를 파이토치에서 구현하는 코드

```
class Net(torch.nn.Module):
    def __init__(self, n_feature, n_hidden, n_output):
        super(Net, self).__init__()
        self.hidden = torch.nn.Linear(n_feature, n_hidden) #은닉층
        self.relu = torch.nn.ReLU(inplace=True)
        self.out = torch.nn.Linear(n_hidden, n_output) #출력층
        self.softmax = torch.nn.Softmax(dim=n_output)
    def forward(self, x):
        x = self.hidden(x)
        x = self.relu(x) #은닉층을 위한 렐루 활성화 함수
        x = self.out(x)
```

```
x = self.softmax(x) # 출력층을 위한 소프트맥스 활성화 함수
return x
```

손실 함수

학습을 통해 얻은 데이터의 추정치가 실제 데이터와 얼마나 차이 나는지 평가하는 지표

평균 제곱 오차

실제 값과 예측 값의 차이(error)를 제곱하여 평균을 낸 것(MSE)

- 값이 작을수록 예측력이 좋음
- 회귀에서 손실 함수로 주로 사용

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$\left(\begin{array}{l} \hat{y}_i: \text{신경망의 출력(신경망이 추정한 값)} \\ y_i: \text{정답 레이블} \\ i: \text{데이터의 차원 개수} \end{array} \right)$

평균 제곱 오차를 구하는 수식

```
#파이토치 사용법
import torch

loss_fn = torch.nn.MSELoss(reduction='sum')
y_pred = model(y)
loss = loss_fn(y_pred,y)
```

크로스 엔트로피 오차

- 분류 문제에서 원-핫 인코딩을 했을 때만 사용하는 오차 계산법
- 경사 하강법 과정에서 학습이 지역 최소점에서 멈출 수 있음 → 이를 방지하고자 자연 상수 e에 반대되는 자연 로그를 모델의 출력 값에 취함

$$CrossEntropy = - \sum_{i=1}^n y_i \log \hat{y}_i$$

$\hat{y}_i$: 신경망의 출력(신경망이 추정한 값)
 y_i : 정답 레이블
 i : 데이터의 차원 개수

크로스 엔트로피를 구하는 수식

#파이토치에서의 사용법

```
loss = nn.CrossEntropyLoss()
```

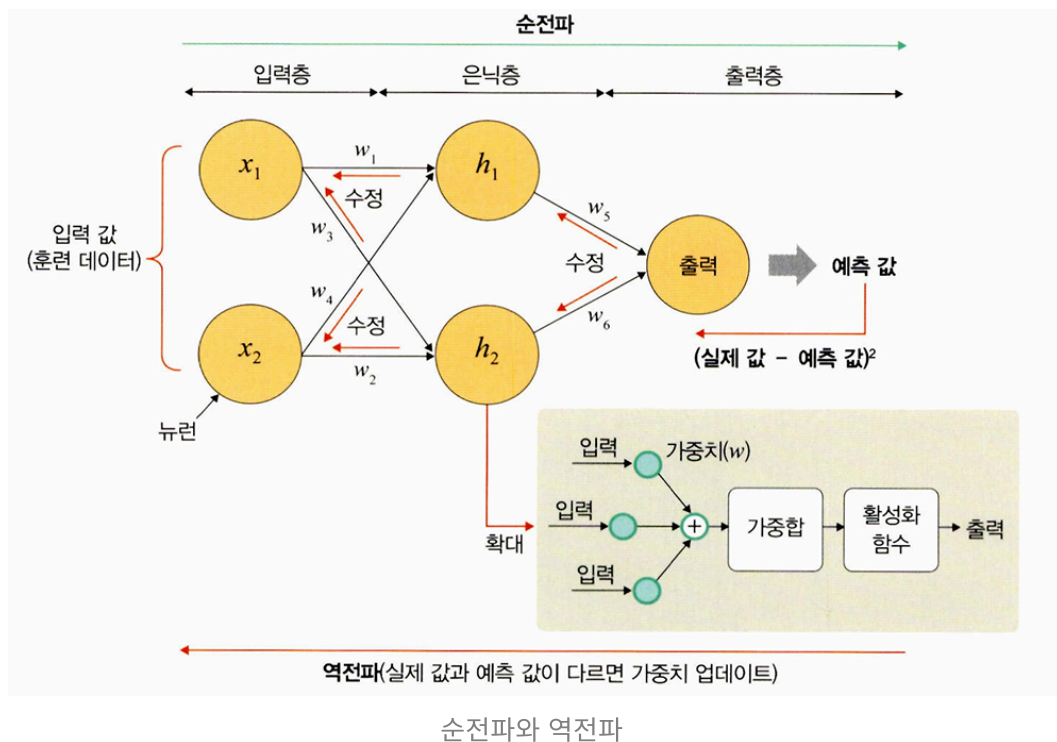
```
input = torch.randn(5,6,requires_grad=True) #평균은 0이고 표준편차가 1인 가우시안 정규분포를 이용하여 숫자 생성
```

```
target = torch.empty(5, dtype=torch.long).random_(5) #dtype torch.float32의 랜덤한 값으로 채워진 텐서를 반환
```

```
output = loss(input, target)
```

```
output.backward()
```

4.2.2 딥러닝 학습



1) 순전파

- 입력 데이터를 네트워크에 넣어 **출력(예측값)** 계산
- 각 층에서:
 - 입력 $\times$ 가중치 + 편향 $\rightarrow$ 활성화 함수 적용
 - 결과를 다음 층으로 전달
- 최종적으로 **출력층에서 예측값**이 나옴
- 손실 함수(Loss)를 통해 예측값과 실제값의 차이 계산

2) 역전파

- 손실값을 **출력층 $\rightarrow$ 은닉층 $\rightarrow$ 입력층**으로 거꾸로 전달
- 각 뉴런/가중치가 오차에 기여한 정도(기울기, Gradient)를 계산
- 수학적으로는 **손실을 각 가중치에 대해 미분**해서 얻음
- 가중치를 오차가 줄어드는 방향으로 업데이트 (경사하강법)

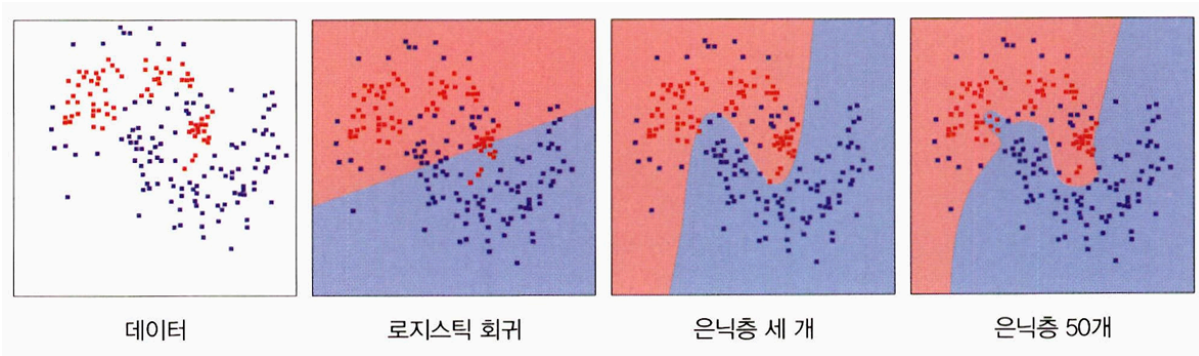
3) 학습 과정 반복

- 순전파로 예측값 계산
- 손실 함수로 오차 측정
- 역전파로 기울기 계산
- 가중치 업데이트 (경사하강법)

$\Rightarrow$ 위 과정을 데이터셋에 대해 여러 번 반복하면서 오차 최소화

4.2.3 딥러닝의 문제점과 해결 방안

딥러닝의 핵심은 활성화 함수가 적용된 여러 은닉층을 결합하여 비선형 영역을 표현하는 것



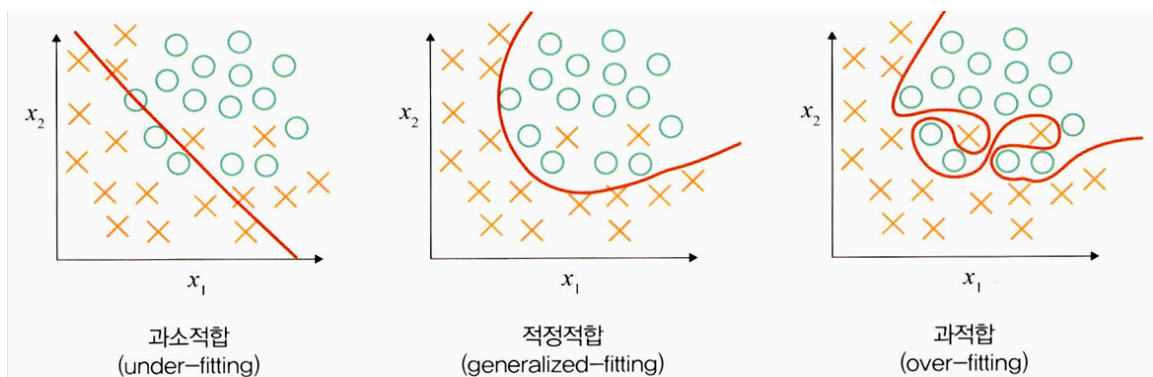
⇒ 활성화가 적용된 은닉층 개수가 많을수록 데이터 분류가 잘 됨

그러나 은닉층이 많을수록 다음 세 가지의 문제점이 생긴다.

1) 과적합 문제 발생

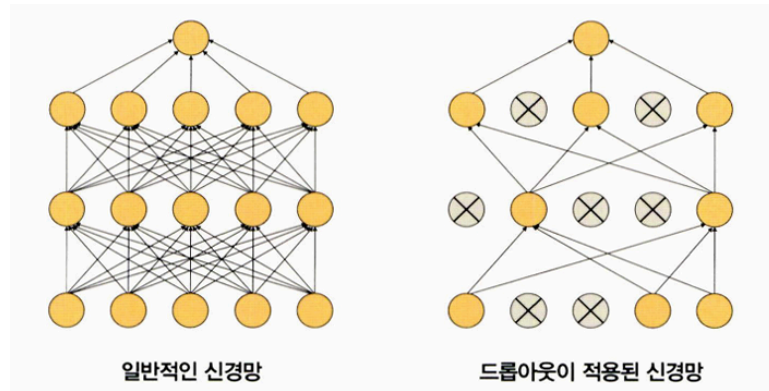
훈련 데이터를 과하게 학습해서 발생

- 오차는 감소 but, 검증 데이터/실제데이터에 대해서는 오차가 증가



과적합 해결 방안 : 드롭아웃(dropout)

- 학습 과정 중 임의로 일부 노드들을 학습에서 제외



#파이토치에서 드롭아웃을 구현하는 예시 코드

```
class DropoutModel(torch.nn.Module):
```

```
    def __init__(self):
```

```
        super(DropoutModel, self).__init__()
```

```
        self.layer1 = torch.nn.Linear(784, 1200)
```

```
        self.dropout1=torch.nn.Dropout(0.5) #50%의 노드를 무작위로 선택하여 사용
```

X

```
        self.layer2 = torch.nn.Linear(1200, 1200)
```

```
        self.dropout2=torch.nn.Dropout(0.5)
```

```
        self.layer3 = torch.nn.Linear(1200, 10)
```

```
    def forward(self, x):
```

```
        x=F.relu(self.layer1(x))
```

```
        x=self.dropout1(x)
```

```
        x=F.relu(self.layer2(x))
```

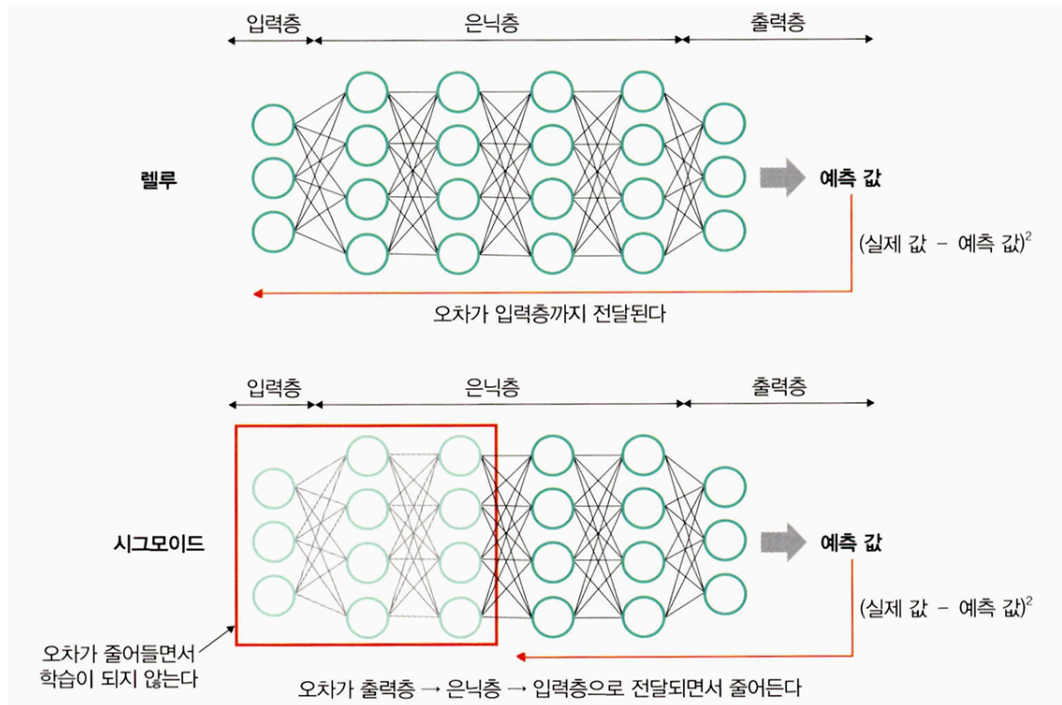
```
        x=self.dropout2(x)
```

```
        return self.layer3(x)
```

2) 기울기 소멸 문제 발생

출력층에서 은닉층으로 전달되는 오차가 크게 줄어들어 학습이 되지 않는 현상

- 은닉층이 많은 신경망에서 주로 발생



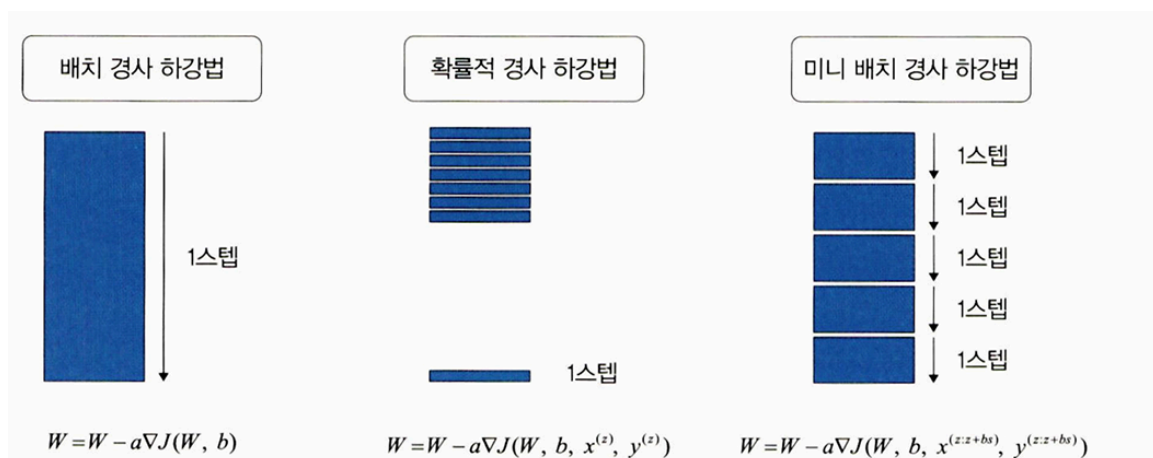
- 해결 방법: 시그모이드, 하이퍼볼릭 탄젠트 대신 **렐루 활성화 함수** 사용

3) 성능이 나빠지는 문제 발생

경사하강법은 손실 함수의 비용이 최소가 되는 지점을 찾을 때까지 기울기가 낮은 쪽으로 계속 이동시키는 과정을 반복하는데, 이때 발생 한다.

- 개선 방안: 확률적 경사 하강법, 미니 배치 경사 하강법

경사하강법



- 배치 경사 하강법

: 전체 데이터셋에 대한 오류를 구한 후 기울기를 한 번만 계산하여 파라미터 업데이트

⇒ 전체 훈련 데이터셋에 대한 가중치를 편미분하는 방법

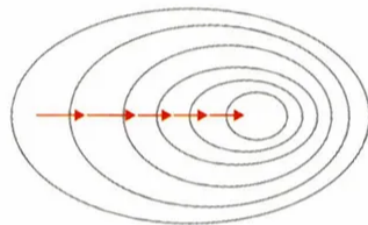
- 단점: 한 스텝에 모든 훈련 데이터셋 사용 → 학습 시간이 오래 걸림

이를 개선한 것이 확률적 경사 하강법

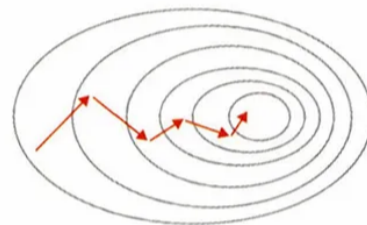
- 확률적 경사 하강법

: 임의로 선택한 데이터에 대해 기울기를 계산하는 방법

- 장점: 적은 데이터 사용 → 빠른 계산 가능
- 단점: 파라미터 변경 폭이 불안정, 때로는 정확도가 더 낮을 수도 있음



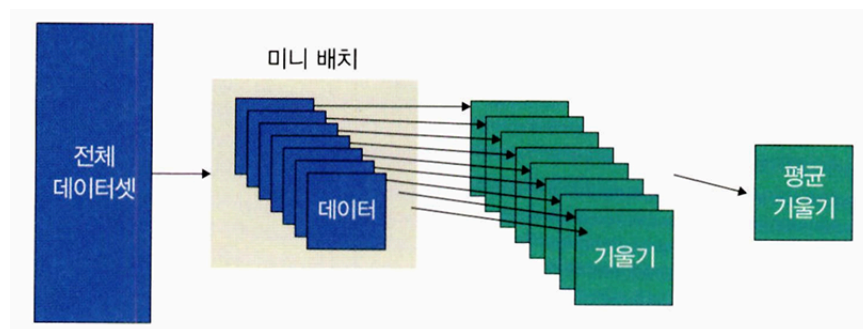
배치 경사 하강법



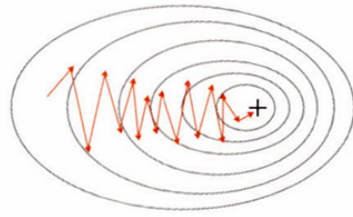
확률적 경사 하강법

- 미니 배치 경사 하강법

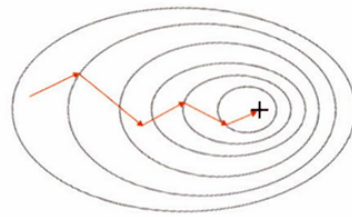
: 전체 데이터셋을 미니 배치 여러 개로 나누고, 미니 배치 한 개마다 기울기를 구한 후 그것의 평균 기울기를 이용하여 모델을 업데이트 하는 방법



- 장점: 전체 데이터 계산 보다 빠르며, 확률적 경사 하강법보다 안정적 → 실제 가장 많이 씀



확률적 경사 하강법

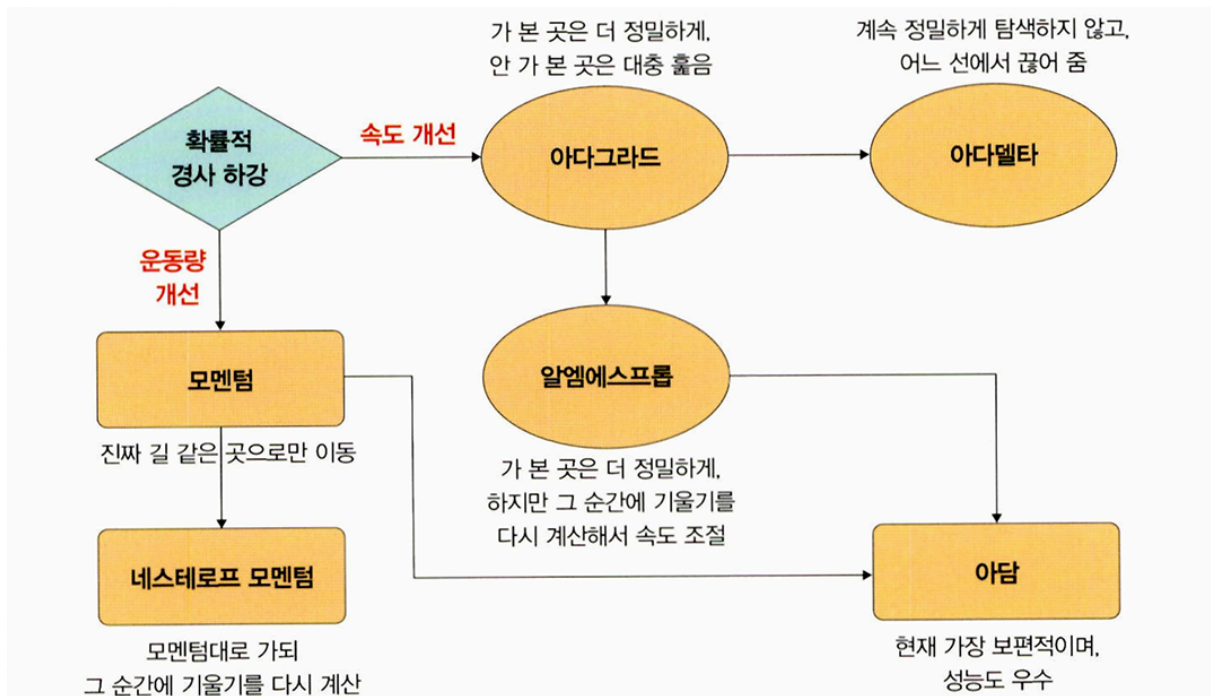


미니 배치 경사 하강법

```
#파이토치에서 구현하는 예시코드
class CustomDataset(Dataset):
    def __init__(self):
        self.x_data = [[1,2,3],[4,5,6],[7,8,9]]
        self.y_data = [[12],[18],[11]]
    def __len__(self):
        return len(self.x_data)
    def __getitem__(self, idx):
        x=torch.FloatTensor(self.x_data[idx])
        y=torch.FloatTensor(self.y_data[idx])
        return x,y
dataset = CustomDataset()
dataloader = DataLoader(
    dataset, #데이터셋
    batch_size=2, #미니 배치 크기로 2의 제공수를 사용하겠다는 의미
    shuffle=True #데이터를 불러올 때마다 랜덤으로 섞어서 가져옴
)
```

옵티마이저

확률적 경사 하강법의 파라미터 변경 폭이 불안정한 문제를 해결하기 위해 학습 속도와 운동량을 조정하는 옵티마이저를 적용할 수 있다



속도를 조정하는 방법

[아다그라드(Adagrad, Adaptive gradient)]

변수(가중치)의 업데이트 횟수에 따라 학습률을 조정하는 방법

- 많이 변화하지 않는 변수들의 학습률은 크게 하고, 많이 변화하는 변수는 작게
⇒ 많이 변화한 변수는 최적 값에 근접했을 거라 가정하여 작은 크기로 이동하며 세밀하게 조정

$$w(i+1) = w(i) - \frac{\eta}{\sqrt{G(i) + \epsilon}} \nabla E(w(i))$$

$$G(i) = G(i-1) + (\nabla E(w(i)))^2$$

- 파라미터마다 다른 학습률을 주기 위한 함수 G(이전 G값의 누적)
- 기울기가 크면 G값이 ↑, 학습률은 ↓

- 파이토치에서의 구현

```
optimizer = torch.optim.Adagrad(model.parameters(), lr=0.01) #학습률 기본값은 1e-2
```

- 단점: 기울기가 0에 수렴 → 알엠에스프롭 사용

[아다델타(Adadelta, Adaptive delta)]

아다그라드에서 G값이 커짐에 따라 학습이 멈추는 문제를 해결하기 위해 등장

- 아다그라드 수식에서 학습률을 D함수(가중치의 변화량 크기를 누적한 값)로 변환
→ 학습률 파라미터 필요 X

$$\begin{aligned}w(i+1) &= w(i) - \frac{\sqrt{D(i-1)+\varepsilon}}{\sqrt{G(i)+\varepsilon}} \nabla E(w(i)) \\G(i) &= \gamma G(i-1) + (1-\gamma)(\nabla E(w(i)))^2 \\D(i) &= \gamma D(i-1) + (1-\gamma)(\Delta(w(i)))^2\end{aligned}$$

- 파이토치에서의 구현

```
optimizer = torch.optim.Adadelta(model.parameters(), lr=1.0) # 학습률 기본값 1.0
```

[알엠에스프롭(RMSProp)]

아다그라드의 G(i)값이 무한히 커지는 것을 방지하고자 제안됨

- 아다그라드에서 학습이 안 되는 문제 해결을 위해 G함수에 감마 추가
⇒ G값이 너무 크면 학습률이 작아져 학습이 안 될 수도, 사용자가 감마값을 이용해 학습률 크기를 비율로 조정할 수 있게 함

$$\begin{aligned}w(i+1) &= w(i) - \frac{\eta}{\sqrt{G(i)+\varepsilon}} \nabla E(w(i)) \\G(i) &= \gamma G(i-1) + (1-\gamma)(\nabla E(w(i)))^2\end{aligned}$$

- 파이토치에서 구현

```
optimizer = torch.optim.RMSprop(model.parameters(), lr=0.01) # 학습률 기본값은 1e-2
```

운동량을 조정하는 방법

[모멘텀(Momentum)]

매번 기울기를 구하지만, 가중치를 수정하기 전에 이전 수정 방향(+,-)을 참고하여 같은 방향으로 일정한 비율만 수정하는 방법

- 장점: 수정이 양(+) 방향과 음(-) 방향으로 순차적으로 일어나는 지그재그 현상이 줄어들어, 관성 효과
- 단점: 멈추어야 할 시점에서도 관성에 의해 훨씬 멀리 갈 수 있음
- SGD 와 함께 사용

$$w(i+1) = w(i) - \eta \nabla E(w(i))$$

◦ 기울기 크기와 반대 방향만큼 가중치 업데이트

- SGD 모멘텀

$$\begin{aligned} w(i+1) &= w(i) - v(i) \\ v(i) &= \gamma v(i-1) + \eta \nabla E(w(i)) \end{aligned}$$

◦ 기울기를 속도로 대체하여, 이전 속도의 일정 부분만 반영

◦ 이전에 학습했던 속도와 현재 기울기를 반영해서 가중치를 구함

- 파이토치에서 구현

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
```

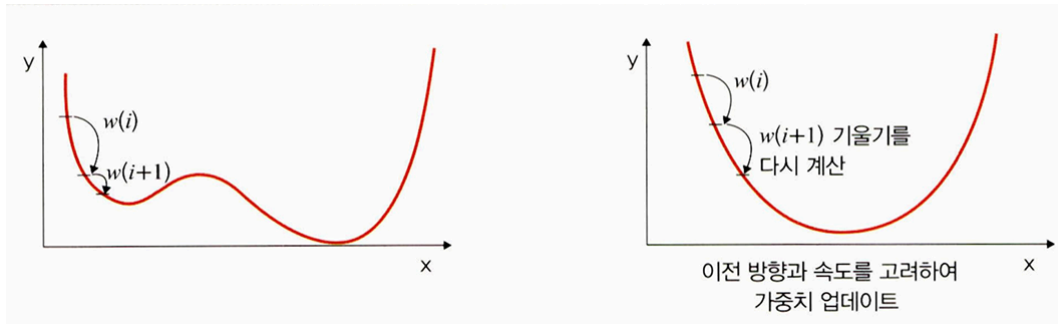
[네스테로프 모멘텀(Nesterov Accelerated Gradient, NAG)]

모멘텀 값이 적용된 지점에서 기울기 값 계산

- 모멘텀의 단점 극복 → 모멘텀의 이점인 빠른 속도 + 멈추어야 할 적절한 시점에서 제동
- 속도 구하는 과정에서 차이: 이전에 학습했던 속도와 현재 기울기에서 이전 속도를 뺀 변화량을 반영해서 가중치를 구함

$$\begin{aligned} w(i+1) &= w(i) - v(i) \\ v(i) &= \gamma v(i-1) + \eta \nabla E(w(i) - \gamma v(i-1)) \end{aligned}$$

- 모멘텀 Vs. 네스테로프 모멘텀



- 파이토치에서 구현

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9, nesterov=True) #nesterov 기본값은 False
```

속도와 운동량에 대한 혼용 방법

[아담(Adam, Adaptive Moment Estimation)]

모멘텀과 알엠에스프롭의 장점을 결합한 경사 하강법

- 알엠에스프롭의 특징인 기울기의 제곱을 지수 평균한 값과 모멘텀 특징인 $v(i)$ 활용

$$w(i+1) = w(i) - \frac{\eta}{\sqrt{G(i) + \epsilon}} v(i)$$

$$G(i) = \gamma_2 G(i-1) + (1 - \gamma_2)(\nabla E(w(i)))^2$$

$$v(i) = \gamma_1 v(i-1) + \eta \nabla E(w(i))$$

⇒ 알엠에스프롭의 G함수와 모멘텀 $v(i)$ 를 사용하여 가중치 업데이트

- 파이토치에서 구현

```
optimizer=torch.optim.Adam(model.parameters(),lr=0.01) #학습률 기본값 1e-3
```

4.2.4 딥러닝을 사용할 때 이점

특성 추출

데이터별로 어떤 특징을 갖고 있는지 찾아내고, 이를 토대로 벡터로 변환하는 작업

- 딥러닝 이전의 머신러닝 알고리즘은 특성 추출이 복잡하며 데이터의 전문 지식이 필요
- 딥러닝은 데이터 특성을 잘 잡아내고자 은닉층을 깊게 쌓는 방식으로 파라미터를 늘림

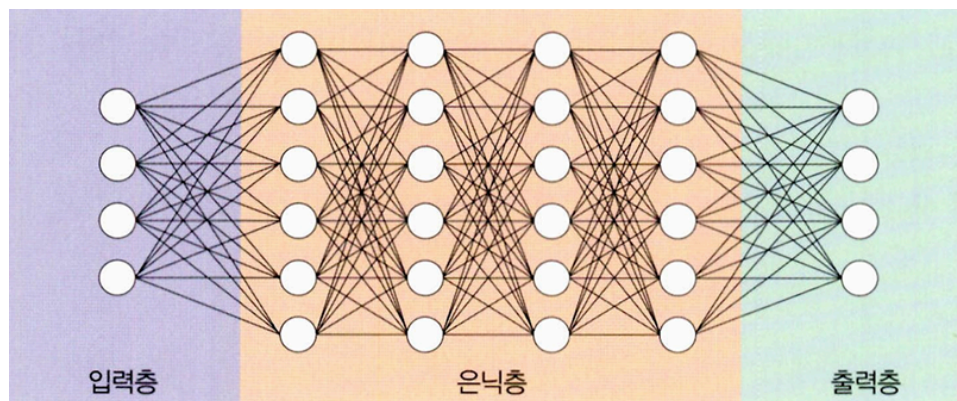
빅데이터의 효율적 활용

확보된 데이터가 많을수록 성능이 향상됨

4.3 딥러닝 알고리즘

딥러닝 알고리즘은 심층 신경망을 공통적으로 사용

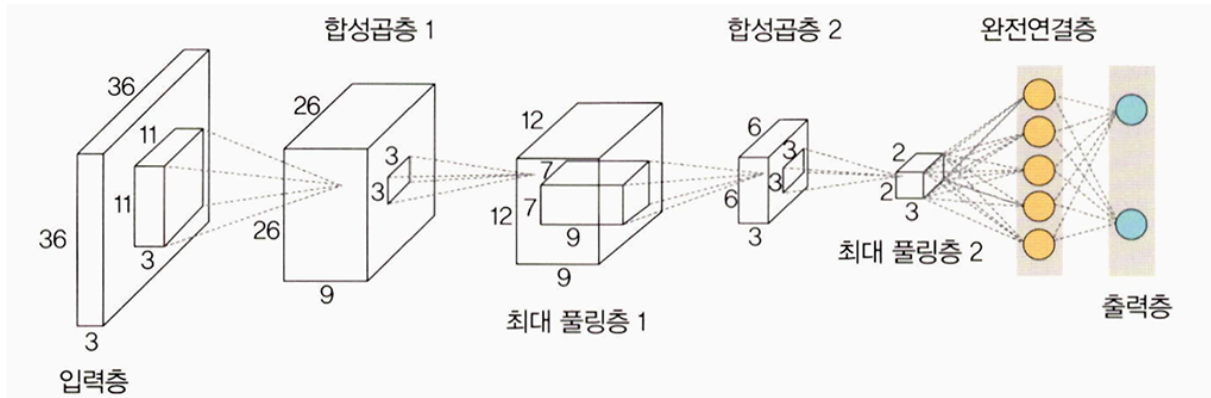
4.3.1 심층 신경망(DNN)



입력층과 출력층 사이에 다수의 은닉층을 포함하는 인공 신경망

- 다수의 은닉층 추가로 별도의 트릭 없이 비선형 분류 가능
- 장점: 다양한 비선형적 관계를 학습할 수 있음
- 단점: 연산량이 많고 기울기 소멸 문제 등 발생 가능
 - 해결 방법: 드롭아웃, 렐루 함수, 배치 정규화 등을 적용

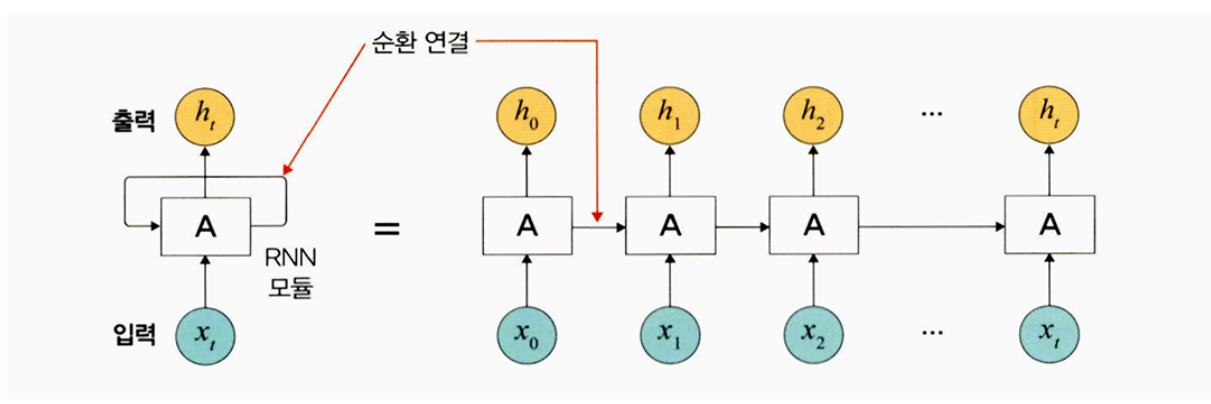
4.3.2 합성곱 신경망(Convolutional Neural Network, CNN)



합성곱층과 풀링층을 포함하는 이미지 처리 성능이 좋은 인공 신경망 알고리즘

- 영상 및 사진이 포함된 이미지 데이터에서 객체/객체 위치 탐색에 유용
- 객체, 얼굴, 장면을 인식하기 위해 패턴을 찾는 데 특히 유용
- 대표적인 CNN: LeNet-5, AlexNet, VGG, GoogLeNet, ResNet 등
- 기존 신경망과의 차별성
 - 각 층의 입출력 형상 유지
 - 이미지의 공간 정보 유지, 인접 이미지와 차이가 있는 특징 효과적으로 인식
 - 복수 필터로 이미지의 특징 추출 및 학습
 - 추출한 이미지 특징을 모으고 강화하는 풀링층
 - 필터를 공유 파라미터로 사용→학습 파라미터가 매우 적음

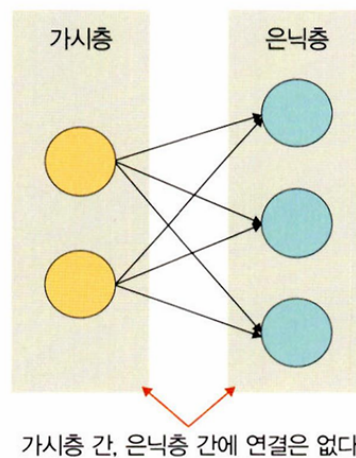
4.3.3 순환 신경망(Recurrent Neural Network, RNN)



시계열 데이터(음악, 영상 등) 같은 시간 흐름에 따라 변화하는 데이터를 학습하기 위한 인공 신경망

- '순환': 자기 자신을 참조 → 현재 결과가 이전 결과와 연관이 있단 의미
- RNN 특징
 - 시간성을 가진 데이터가 많음
 - 시간성 정보를 이용하여 데이터의 특징을 잘 다룸
 - 시간에 따라 내용이 변함 → 데이터는 동적이며 길이는 가변적
 - 매우 긴 데이터 처리 연구가 활발히 진행
- 단점: 기울기 소멸 문제로 학습이 제대로 되지 않는 문제
→ 해결 방법 : LSTM(Long-Short Term Memory) 사용
- 자연어 처리 분야(언어 모델링, 텍스트 생성, 자동 번역, 음성 인식, 이미지 캡션 생성 등)

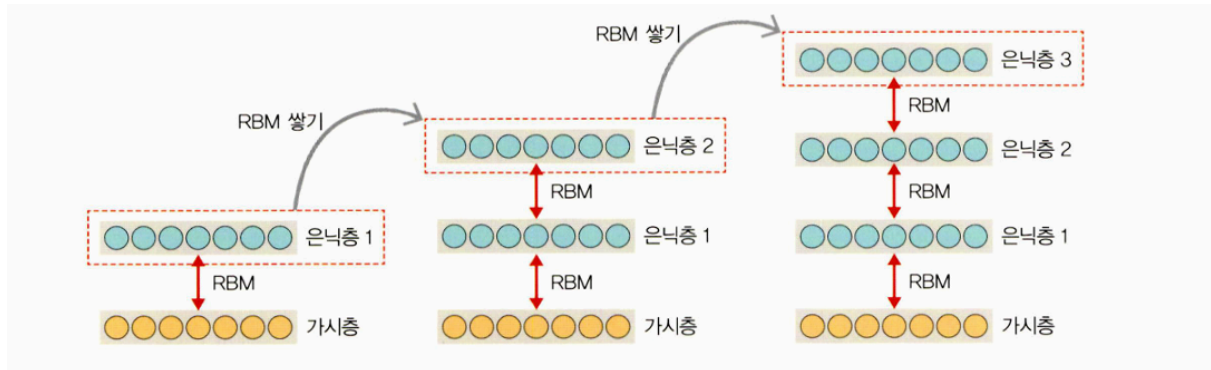
4.3.4 제한된 볼츠만 머신(Restricted Boltzmann machine, RBM)



가시층과 은닉층으로 구성되어 있으며, 가시층은 은닉층과만 연결

- 가시층과 가시층, 은닉층과 은닉층 사이에는 연결이 없음
- RBM의 특징
 - 차원 감소, 분류, 선형 회귀 분석, 협업 필터링, 특성 값 학습, 주제 모델링에 사용
 - 기울기 소멸 문제를 해결하기 위해 사전 학습 용도로 활용 가능
 - 심층 신뢰 신경(DBN)의 요소로 활용

4.3.5 심층 신뢰 신경망(Deep Belief Network, DBN)



입력층과 은닉층으로 구성된 RBM을 블록처럼 여러 층으로 쌓은 형태로 연결된 신경망

- 레이블이 없는 데이터에 대한 비지도 학습 가능
- 부분적인 이미지에서 전체를 연상하는 일반화와 추상화 과정을 구현할 때 유용
- 학습 절차
 1. 가시층과 은닉층 1에 RBM을 사전 훈련
 2. 첫 번째 층 입력 데이터와 파라미터를 고정하여 두 번째 층 RBM을 사전 훈련
 3. 원하는 층 개수만큼 RBM을 쌓아 올려 전체 DBN 완성
- DBN 특징
 - 순차적으로 DBN을 학습시켜 계층적 구조 생성
 - 비지도 학습으로 학습
 - 위로 올라갈수록 추상적 특성을 추출
 - 학습된 가중치를 다층 퍼셉트론의 가중치 초기값으로 사용

4.4 우리는 무엇을 배워야 할까?

- 데이터를 활용하여 얻고자 하는 것에 따라 머신러닝/딥러닝을 선택
- 딥러닝 도구의 활용법을 배워보자!